

INDIAN INSTITUTE OF TECHNOLOGY PALAKKAD

DS2020 INTRODUCTION TO ARTIFICIAL INTELLIGENCE

Lab 4

Students:

Abhishek Iyer (112401002)

Instructor:

Dr. Narayanan C Krishnan

April 7, 2026



INDIAN INSTITUTE
OF TECHNOLOGY
PALAKKAD

Contents

1	Background	2
a)	Boolean Satisfiability (SAT)	2
b)	Sudoku Rules	2
2	Propositional Variable Encoding	2
a)	Variable Definition	2
b)	Integer Identifier	3
3	CNF Constraint Encoding	3
a)	Constraint 1 — Each Cell Has At Least One Value	3
b)	Constraint 2 — Each Cell Has At Most One Value	3
c)	Constraint 3 — Row and Column Uniqueness	4
d)	Constraint 4 — Box Uniqueness	5
e)	Constraint 5 — Pre-Filled (Given) Cells	5
4	Complete CNF Formula	5
a)	Clause Count Summary	5
5	Solving and Decoding	6
a)	SAT Solver Invocation	6
b)	Decoding the Solution	6
6	Example: A Single Cell	6
7	Conclusion	7

1 Background

a) Boolean Satisfiability (SAT)

A Boolean satisfiability problem asks: given a propositional formula over a set of Boolean variables, does there exist a truth assignment to those variables that makes the formula true?

Definition 1 (Conjunctive Normal Form) A formula is in **Conjunctive Normal Form (CNF)** if it is a conjunction ($\wedge$) of clauses, where each clause is a disjunction ($\vee$) of literals, and a literal is either a variable x_i or its negation $\neg x_i$:

$$\Phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m, \quad C_k = \ell_{k,1} \vee \ell_{k,2} \vee \cdots \vee \ell_{k,n_k}.$$

By the Tseitin transformation (and the simpler direct encoding used here), every finite constraint satisfaction problem can be translated to CNF, which is the input format expected by pycosat.

b) Sudoku Rules

A standard Sudoku puzzle is played on a 9×9 grid divided into nine 3×3 sub-boxes. A valid solution must satisfy:

- R1.** Every cell contains exactly one digit from $\{1, \dots, 9\}$.
- R2.** Every **row** contains each digit exactly once.
- R3.** Every **column** contains each digit exactly once.
- R4.** Every **3×3 box** contains each digit exactly once.
- R5. Pre-filled** (given) cells are fixed to their stated values.

2 Propositional Variable Encoding

a) Variable Definition

We introduce one Boolean variable for every possible (row, column, digit) triple:

Definition 2 (Propositional variable) For row $i \in \{0, \dots, 8\}$, column $j \in \{0, \dots, 8\}$, and digit $d \in \{1, \dots, 9\}$, define:

$$x_{i,j,d} = \text{true} \iff \text{cell}(i, j) \text{ contains digit } d.$$

The total number of variables is $9 \times 9 \times 9 = 729$.

b) Integer Identifier

The solver (pysosat) works with non-zero integer literals. Variables are mapped to positive integers via the bijection implemented in the code:

$$\text{var}(i, j, d) = 81i + 9j + d \in \{1, \dots, 729\}.$$

In Python:

```
1 def var(row, col, digit):
2     return 81 * row + 9 * col + digit
```

A positive literal $+v$ asserts $x = \text{true}$; a negative literal $-v$ asserts $x = \text{false}$.

3 CNF Constraint Encoding

Each of the five Sudoku rule families is translated into a set of CNF clauses. We use the standard **direct (pairwise) encoding** for the “at-most-one” constraints.

a) Constraint 1 — Each Cell Has At Least One Value

Intuition. Cell (i, j) cannot be empty.

For every cell (i, j) , add the single at-least-one clause:

$$\bigvee_{d=1}^9 x_{i,j,d}$$

This is a single 9-literal clause per cell, yielding $9 \times 9 = 81$ clauses total.

In Python:

```
1 for i in range(9):
2     for j in range(9):
3         clauses.append([var(i, j, d) for d in range(1, 10)])
```

b) Constraint 2 — Each Cell Has At Most One Value

Intuition. A cell cannot simultaneously contain two different digits.

For every cell (i, j) and every pair of distinct digits $d < d'$, add:

$$\neg x_{i,j,d} \vee \neg x_{i,j,d'}$$

This is the pairwise at-most-one encoding. It forbids any two digits being true at the same cell. The number of clauses is:

$$9 \times 9 \times \binom{9}{2} = 81 \times 36 = 2,916.$$

In Python:

```

1 for i in range(9):
2     for j in range(9):
3         for d in range(1, 10):
4             for d1 in range(d + 1, 10):
5                 clauses.append([-var(i, j, d), -var(i, j, d1)])

```

Constraints 1 and 2 together constitute the **exactly-one** encoding for cell values:

$$\forall i, j: \left(\bigvee_{d=1}^9 x_{i,j,d} \right) \wedge \left(\bigwedge_{1 \leq d < d' \leq 9} \neg x_{i,j,d} \vee \neg x_{i,j,d'} \right).$$

c) Constraint 3 — Row and Column Uniqueness

Intuition. No digit may appear twice in the same row, and no digit may appear twice in the same column.

The helper function `make_diff` encodes all-different over a list of cells using the pairwise inequality approach: for every two distinct cells in the collection and for every digit d , it is not the case that both cells contain d :

$$\forall c \neq c' \in S, \forall d: \neg x_{c,d} \vee \neg x_{c',d}$$

where S is the set of cells in the row (or column).

```

1 def make_diff(cells):
2     for cell in cells:
3         for cell1 in cells:
4             if cell != cell1:
5                 if cell[0] < cell1[0] or cell[1] < cell1[1]:
6                     for d in range(1, 10):
7                         clauses.append(
8                             [-var(cell[0], cell[1], d),
9                              -var(cell1[0], cell1[1], d)]
10                        )

```

The guard `cell[0] < cell1[0] or cell[1] < cell1[1]` ensures each unordered pair $\{c, c'\}$ is processed exactly once, avoiding duplicate clauses.

Row uniqueness. For row i , the relevant cells are $\{(i, 0), (i, 1), \dots, (i, 8)\}$. The constraint expands to:

$$\forall i, \forall 0 \leq j < j' \leq 8, \forall d: \neg x_{i,j,d} \vee \neg x_{i,j',d}$$

Column uniqueness. For column j , the relevant cells are $\{(0, j), (1, j), \dots, (8, j)\}$:

$$\forall j, \forall 0 \leq i < i' \leq 8, \forall d: \neg x_{i,j,d} \vee \neg x_{i',j,d}$$

Each of the 9 rows (and 9 columns) contributes $\binom{9}{2} \times 9 = 36 \times 9 = 324$ clauses, giving $2 \times 9 \times 324 = 5,832$ clauses for rows and columns combined.

```

1 for i in range(9):
2     for j in range(9):
3         make_diff([[i, j] for j in range(9)]) # row i
4         make_diff([[j, i] for j in range(9)]) # column i

```

d) Constraint 4 — Box Uniqueness

Intuition. Each 3×3 sub-box must contain every digit exactly once.

The nine boxes are indexed by $(b_r, b_c) \in \{0, 1, 2\}^2$. Box (b_r, b_c) covers cells $\{(3b_r + k_r, 3b_c + k_c) \mid k_r, k_c \in \{0, 1, 2\}\}$.

The same pairwise all-different encoding is applied:

$$\forall (b_r, b_c), \forall \text{ distinct } c, c' \in \text{box}(b_r, b_c), \forall d : \neg x_{c,d} \vee \neg x_{c',d}$$

Each of the 9 boxes contributes 324 clauses, for $9 \times 324 = 2,916$ additional clauses.

```
1 for i in range(3):
2     for j in range(3):
3         make_diff([[3*i + k % 3, 3*j + k // 3] for k in range(9)])
```

Here the index $k \in \{0, \dots, 8\}$ is decoded as $k_r = k \bmod 3$ and $k_c = \lfloor k/3 \rfloor$, traversing all 9 cells of the box.

e) Constraint 5 — Pre-Filled (Given) Cells

Intuition. Cells that are already filled in the puzzle must retain their given values.

For each pre-filled cell (i, j) with given digit $g_{i,j}$, add the unit clause:

$$x_{i,j,g_{i,j}}$$

A unit clause forces the corresponding variable to true, fixing that cell.

```
1 for i in range(9):
2     for j in range(9):
3         if grid[9 * i + j] != '.':
4             clauses.append([var(i, j, int(grid[9 * i + j]))])
```

4 Complete CNF Formula

The complete Boolean formula passed to the SAT solver is the conjunction of all clause families:

$$\Phi_{\text{Sudoku}} = \underbrace{\Phi_{\geq 1}}_{\text{at-least-one}} \wedge \underbrace{\Phi_{\leq 1}}_{\text{at-most-one}} \wedge \underbrace{\Phi_{\text{row}}}_{\text{row diff.}} \wedge \underbrace{\Phi_{\text{col}}}_{\text{col diff.}} \wedge \underbrace{\Phi_{\text{box}}}_{\text{box diff.}} \wedge \underbrace{\Phi_{\text{given}}}_{\text{given cells}}$$

a) Clause Count Summary

Constraint	Formula	Clauses
At-least-one (cell)	9×9	81
At-most-one (cell)	$9 \times 9 \times \binom{9}{2}$	2,916
Row uniqueness	$9 \times \binom{9}{2} \times 9$	2,916
Column uniqueness	$9 \times \binom{9}{2} \times 9$	2,916
Box uniqueness	$9 \times \binom{9}{2} \times 9$	2,916
Given cells	(up to 81, puzzle-dependent)	≤ 81
Total (sans givens)		11,745

5 Solving and Decoding

a) SAT Solver Invocation

The full clause list is passed directly to `pycosat.solve()`:

```
1 solution = pycosat.solve(clauses)
```

`pycosat` implements the DPLL algorithm with unit propagation and backtracking. It returns:

- A list of signed integers — the satisfying assignment — if the formula is satisfiable (SAT).
- The string "UNSAT" if no solution exists.
- The string "UNKNOWN" on timeout.

b) Decoding the Solution

The satisfying assignment is a list of signed integers. A positive integer v in the list means variable v is true, i.e., cell (i, j) contains digit d where $v = 81i + 9j + d$.

To reconstruct the grid, for each cell (i, j) we scan digits $d \in \{1, \dots, 9\}$ and output the unique d for which $\text{var}(i, j, d) \in \text{solution}$:

```
1 for i in range(9):
2     for j in range(9):
3         for d in range(1, 10):
4             if var(i, j, d) in solution:
5                 f.write(str(d))
```

Because the SAT encoding guarantees exactly one d is true per cell, this loop writes exactly one digit per cell, producing the completed 81-character grid.

6 Example: A Single Cell

To cement intuition, consider cell $(0, 0)$ (top-left corner). Suppose it is empty (unknown).

At-least-one clause (one clause, 9 literals):

$$x_{0,0,1} \vee x_{0,0,2} \vee \dots \vee x_{0,0,9}$$

At-most-one clauses (36 clauses, 2 literals each):

$$(\neg x_{0,0,1} \vee \neg x_{0,0,2}), (\neg x_{0,0,1} \vee \neg x_{0,0,3}), \dots, (\neg x_{0,0,8} \vee \neg x_{0,0,9})$$

Row 0 uniqueness clause for digit 5 (one of 36 row clauses for row 0):

$$\neg x_{0,0,5} \vee \neg x_{0,1,5} \quad \wedge \quad \neg x_{0,0,5} \vee \neg x_{0,2,5} \quad \wedge \quad \dots$$

If the puzzle gives cell $(0, 0) = 3$, the unit clause $x_{0,0,3}$ is added, forcing $x_{0,0,3} = \text{true}$ and allowing unit propagation to set all $x_{0,0,d} = \text{false}$ for $d \neq 3$ via the at-most-one clauses.

7 Conclusion

The encoding maps each of the five Sudoku rule families to a natural family of CNF clauses:

Sudoku Rule	CNF Encoding
Cell has a value	At-least-one clause per cell
Cell has one value	Pairwise at-most-one per cell
Row / column uniqueness	Pairwise negation over each row/column
Box uniqueness	Pairwise negation over each 3×3 box
Pre-filled cells	Unit clause per given cell

The resulting formula has 729 variables and approximately 11,745 clauses (plus up to 81 unit clauses for givens), which pycosat resolves in milliseconds even for the hardest published Sudoku puzzles. This approach is elegant because Sudoku's hard combinatorial constraints map directly onto canonical CNF patterns, requiring no auxiliary variables or complex transformations.